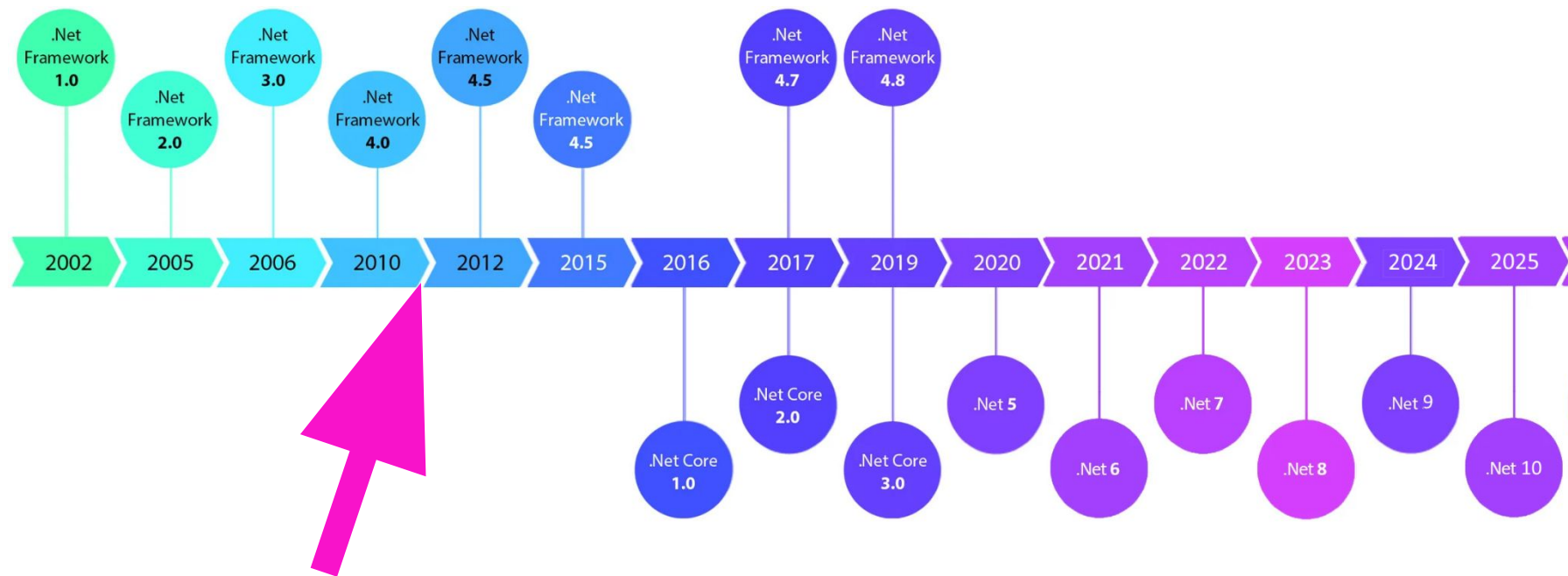


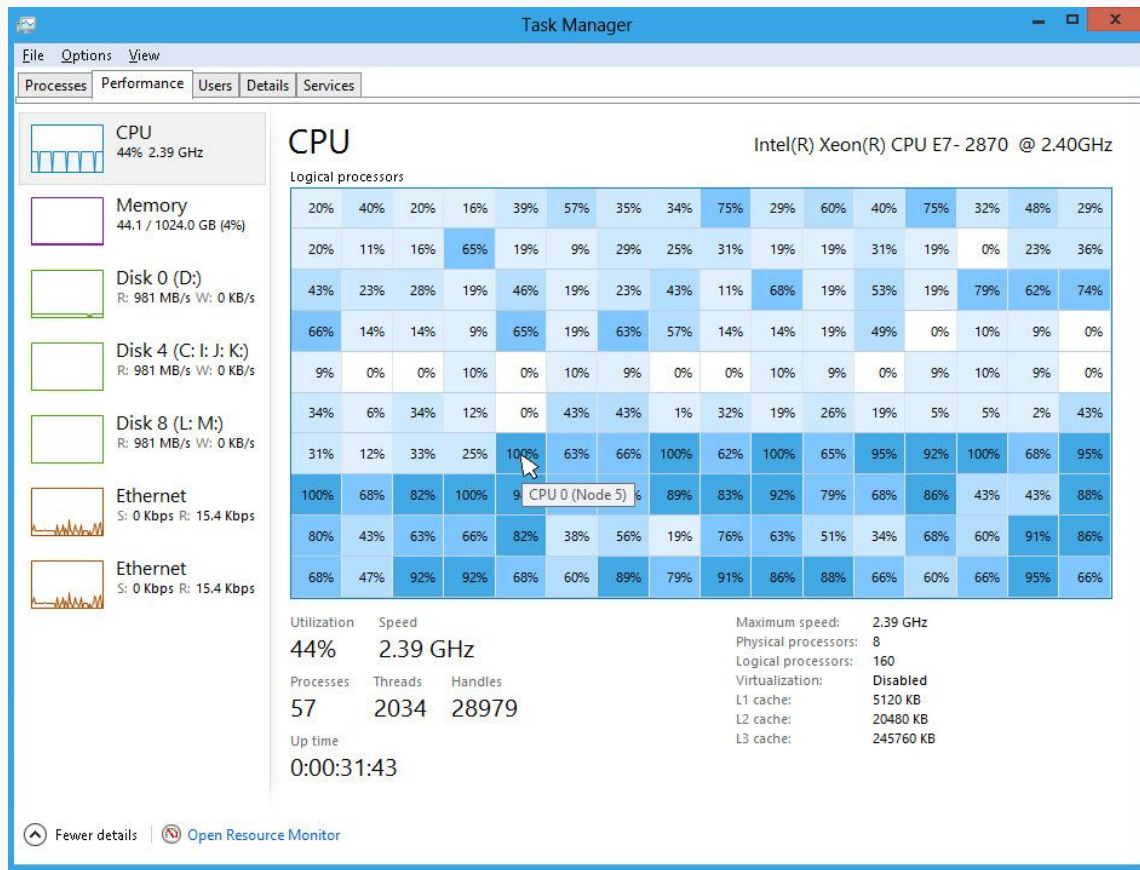


`Async/await` – асинхронность, которую можно читать

Николай Воронин

C#.NET, backend, Altenar





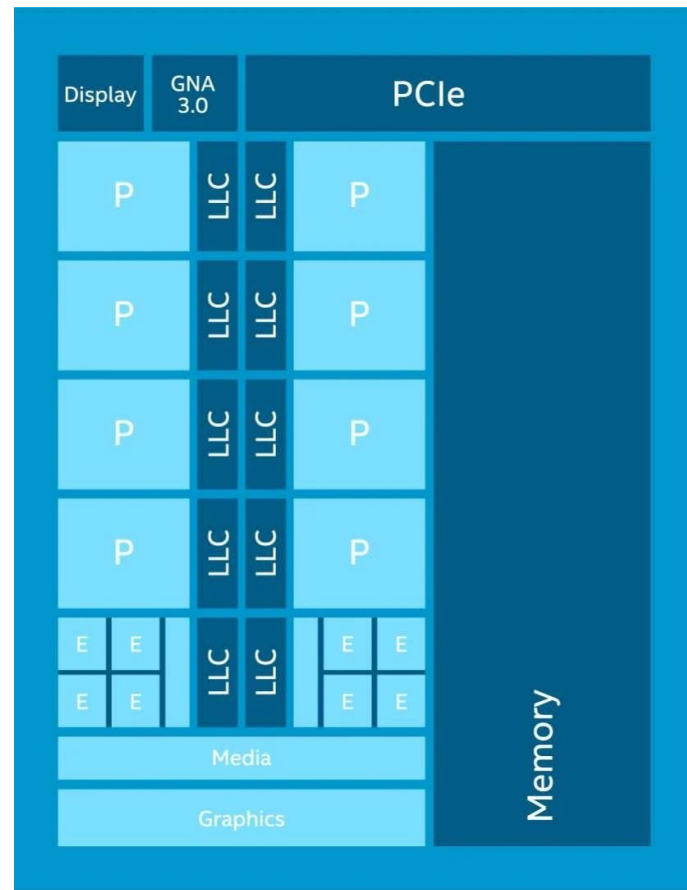
Больше потоков $\neq$ быстрее

- CPU ограничен ядрами
- P/E ядра (Performance / Efficiency)
- Одно ядро – несколько потоков

- Переключение контекста

Конкуренция за ресурсы

Синхронизация



Потоки – дорогие

Параллельно в .NET

- System.Diagnostics.**Process**
- System.**Threading**.Thread
- System.**Threading**.ThreadPool ⚡
- System.**Threading**: Interlocked

Semaphore

Monitor

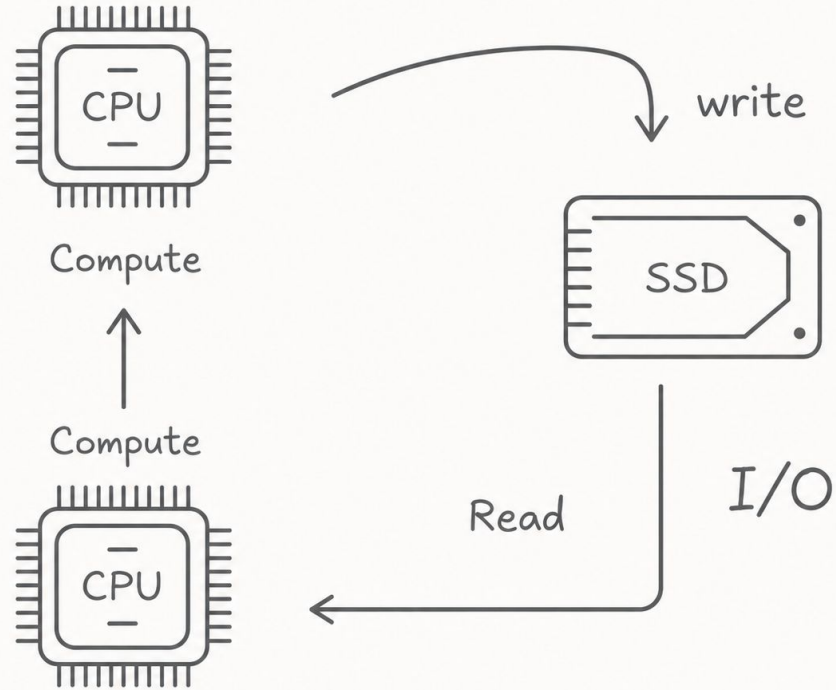
Mutex

...



Как посчитать быстрее?

Параллельно!



Регистры CPU	1 такт
Кэши L1-2-3 (SRAM)	1 – 10 нано сек. (0,000 000 010 сек)
Оперативная память (ОЗУ, DRAM)	50 – 100 нс
SSD (NAND) / Ethernet	10 – 100 микро-сек (0,000 010 сек)
HDD / Ethernet	5 – 10 милли-сек (0,005 сек)
REST, Интернет	50 мс – 0,5 сек, 

База

- Windows

IOCP

OVERLAPPED

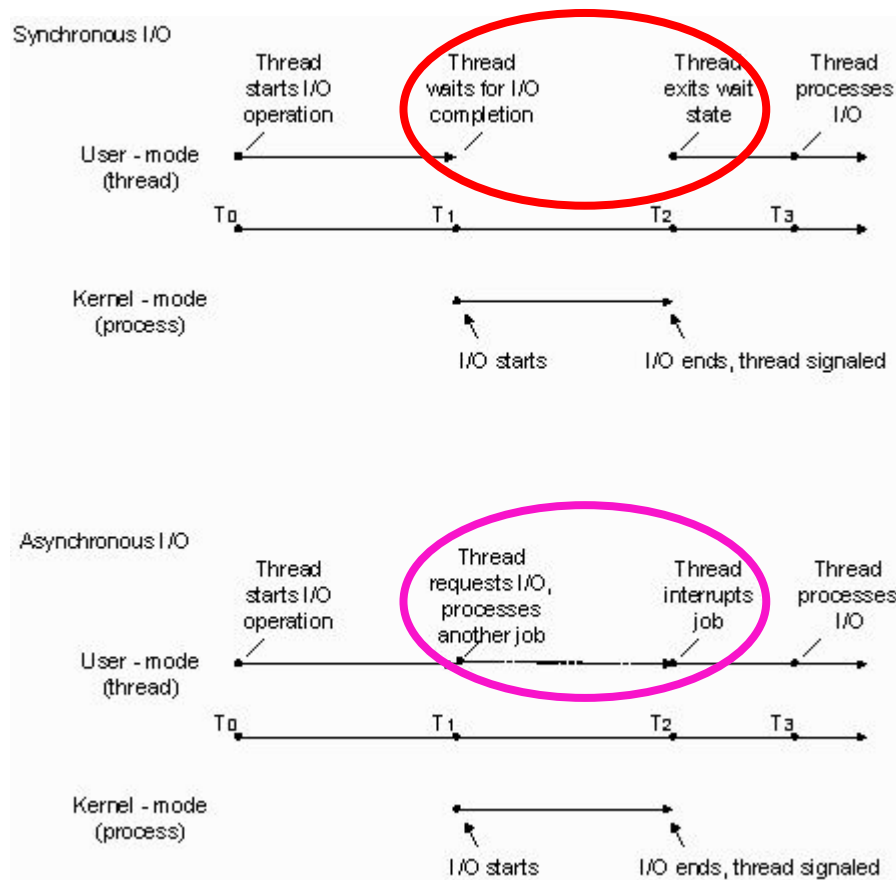
- Linux

AIO

io_uring, epoll

- macOS / BSD

ТОЖЕ ЧТО-ТО ЕСТЬ



Асинхронность —
способ не держать поток

.NET до нашей эры

- **Asynchronous Programming Model**
callbacks, **IAsyncResult**, Begin/End-методы
- **Event-based Asynchronous Pattern**
DoCompleted += / e.Error / e.Cancelled
- **Task-based Asynchronous Pattern**
Task, Unwrap, ContinueWith

APM

EAP

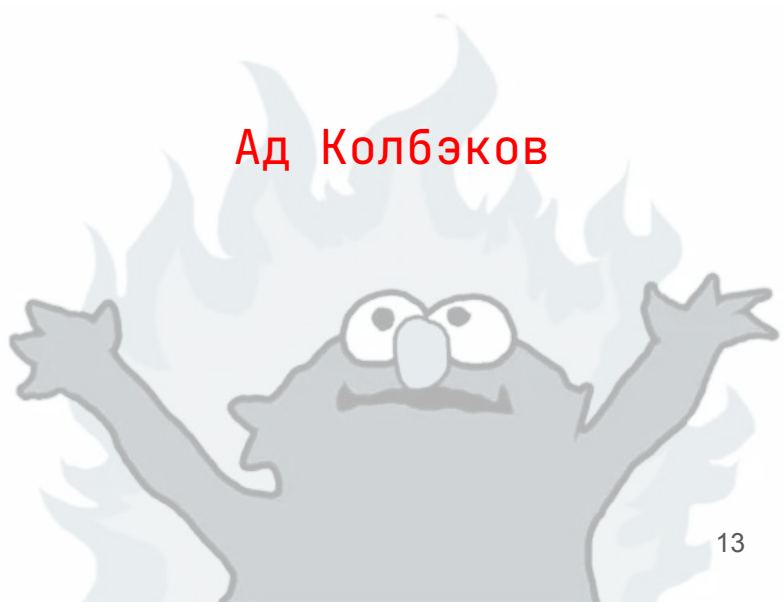
TAP



Asynchronous Programming Model

```
byte[] buffer = new byte[1024];  
FileStream src, dst = File.OpenRead("in.txt");  
FileStream dst = File.Create("out.txt");  
  
src.BeginRead(buffer, 0, buffer.Length, r =>  
{  
    int bytesRead = src.EndRead(r);  
    dst.BeginWrite(buffer, 0, bytesRead, w =>  
    {  
        dst.EndWrite(w);  
        src.Close(); dst.Close();  
    }, null);  
}, null);
```

Ад Колбэков



Event-based Asynchronous Pattern

```
var client = new WebClient();
```

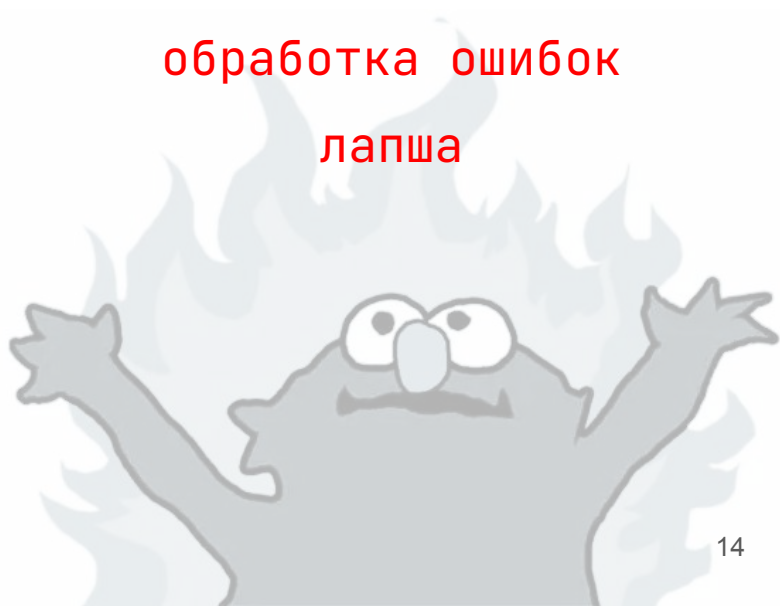
```
client.DownloadCompleted += (s,e) =>
{
    if (e.Error != null) OnError()
    else if (e.Cancelled) OnCancelled()
    else OnResult();
}
```

```
client.DownloadAsync( "https://example.com" );
```

сложно композировать

обработка ошибок

лапша



Task-based Asynchronous Pattern

LoadAsync()

```
.ContinueWith( t => {  
    if ( t.IsFaulted ) { return Task.FromException<Data>( t.Exception ); }  
    return ProcessAsync( t.Result );  
}).Unwrap()  
.ContinueWith( t => {  
    if ( t.IsFaulted ) { return Task.FromException( t.Exception ); }  
    return SaveAsync( t.Result );  
}).Unwrap()  
.ContinueWith( t => {  
    if ( t.IsFaulted ) Log( t.Exception ); else Done();  
});
```



Task<T>

- объект, представляющий асинхронную операцию
ее состояние, результат <T> и возможные ошибки
- используется как единица работы
и точка композиции/синхронизации (WhenAll, ContinueWith)

Future, Promise, Deferred, Coroutine, Fiber, ...

TASK

обещание результата

1 Заказ и вызов асинхронной операции

2 Ожидание завершения I/O операции (поток освобождён)

3 Продолжение после await может выполнить другой поток

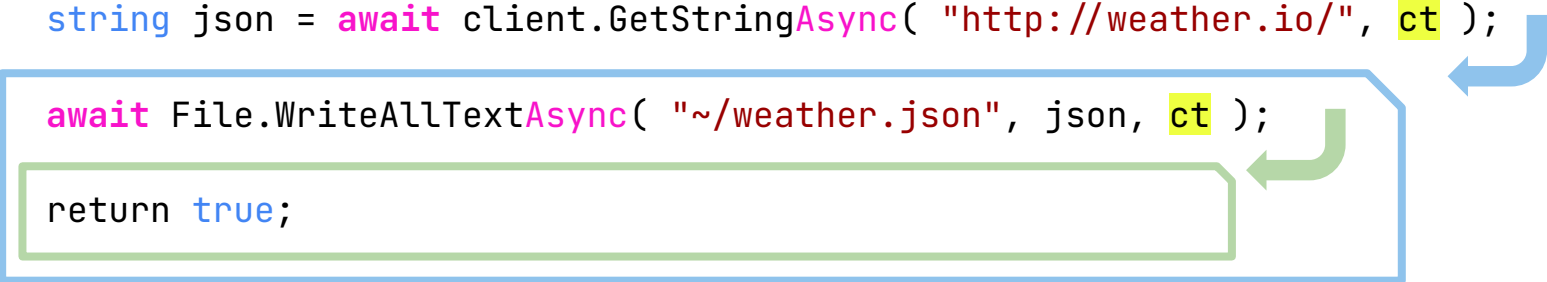


```
A  async Task<bool> FetchWriteAllAsync( CancellationToken ct )  
    {  
        var client = _clientFactory.Create();  
  
B    string json = await client.GetStringAsync( "http://weather.io/", ct );  
  
B    await File.WriteAllTextAsync( "~/weather.json", json, ct );  
  
Γ    return true;  
    }
```

```
Ж  bool success = await FetchWriteAllAsync( cancellationToken );
```



```
A  async Task<bool> FetchWriteAllAsync( CancellationToken ct )  
    {  
        var client = _clientFactory.Create();  
  
        B  string json = await client.GetStringAsync( "http://weather.io/", ct );  
        B  await File.WriteAllTextAsync( "~/weather.json", json, ct );  
        Γ  return true;  
    }  
  
Ж  bool success = await FetchWriteAllAsync( token );
```



Пока **ждем** результат **I/O***
отдаем **поток** для других задач

Планирование “продолжения”

- **ThreadPool** ⚡ – общий пул рабочих потоков, очередь на выполнение
ASP.NET, Console
- **SynchronizationContext** – показывает куда вернуться “продолжению”
WinForms, WPF, Avalonia UI, MAUI, WinUI
– Message Pump, Message Loop, Dispatcher или DispatcherQueue

Кооперативная отмена

`CancellationTokenSource` предоставляет токен отмены `CancellationToken`

```
var tokenSource = new CancellationTokenSource();  
CancellationToken ct = tokenSource.Token;
```

```
tokenSource.Cancel();
```

```
Task<bool> DoSmthAsync( CancellationToken ct )  
{  
    if ( !ct.IsCancellationRequested ) throw Exception...  
    ...  
    ct.ThrowIfCancellationRequested();  
    ...  
}
```

Обработка исключений

Await транслирует исключения в точку вызова метода

```
try {  
    var text = await _file.ReadAsync( cancellationToken );  
}  
catch (IOException) {  
    Debug...  
}  
catch (OperationCanceledException) {  
    ...  
}
```

Async экономит потоки,
увеличивая пропускную способность **I/O**

Parallel тратит потоки,
ускоряя вычисления **CPU**

async2



runtime-async

.NET 11 introduces runtime-native async
(Runtime Async V2)



.GetAwaiter() .GetResult()

- **async/await** обычный способ писать I/O в .NET
- **Task** подход без зоопарка колбэков и событий
- Удобный и лаконичный синтаксис
- Линейный последовательный код
- Обработка ошибок без манипуляций



Эталон для индустрии и других ЯП



.NET 9 Runtime **Async**
Experiment

Async/await – асинхронность, которую можно читать

Deep .NET: Writing async/await from scratch in C#

<https://www.youtube.com/watch?v=R-z2Hv-7nxk>

There Is No Thread

<https://blog.stephencleary.com/2013/11/there-is-no-thread.html>

How Async/Await Really Works in C#

<https://devblogs.microsoft.com/dotnet/how-async-await-really-works/>



Николай Воронин

C#.NET, backend
Altenar

Continuation-passing style

